

**UNIVERSIDAD TECNOLÓGICA NACIONAL**  
Tecnicatura Universitaria en Programación – Modalidad a Distancia

**Programación I**

**Trabajo Práctico Integrador (TPI)**

**Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas**

**Integrantes:**

Luciano Acquaviva, DNI: 44.284.636

Alejo Giannini, DNI: 45.214.115

## Índice

|                                             |    |
|---------------------------------------------|----|
| Índice.....                                 | 2  |
| 1. Marco Teórico .....                      | 3  |
| 1.1. Listas .....                           | 3  |
| 1.2. Diccionarios .....                     | 3  |
| 1.3. Funciones .....                        | 3  |
| 1.4. Estructuras condicionales .....        | 3  |
| 1.5. Ordenamientos .....                    | 4  |
| 1.6. Estadísticas básicas .....             | 4  |
| 1.7. Archivos CSV.....                      | 4  |
| 2. Decisiones Técnicas y Arquitectura ..... | 6  |
| 2.1. Estructura de datos.....               | 6  |
| 2.2. Modularización.....                    | 6  |
| 2.3. Diagrama de flujo.....                 | 7  |
| 2.4. Capturas de pantalla.....              | 7  |
| 3. Dificultades y Conclusiones .....        | 11 |
| 3.1. Dificultades encontradas .....         | 11 |
| 3.2. Conclusiones .....                     | 11 |
| 4. Bibliografía / Webgrafía.....            | 12 |
| 5. Enlaces .....                            | 13 |

# 1. Marco Teórico

En esta sección se explican los conceptos teóricos de Programación I, aplicados en el desarrollo del presente Trabajo Práctico Integrador, relacionando cada uno con su implementación concreta en el sistema de gestión de países.

## 1.1. Listas

Una lista es una estructura que permite guardar varios elementos juntos, uno detrás de otro, accediendo a cada uno según su posición. Lo importante es que esos elementos pueden ser de cualquier tipo, incluso otras estructuras como diccionarios. En nuestro proyecto usamos una lista llamada ***países***, que contiene a todos los países cargados. Cada elemento de esa lista es, a su vez, un diccionario con los datos de un país.

## 1.2. Diccionarios

Un diccionario es como una lista, pero en vez de acceder a los datos por posición (0, 1, 2...) se accede por un nombre o "clave". En nuestro caso, cada país es un diccionario con las claves nombre, población, superficie y continente. Esto hace que sea mucho más claro trabajar con los datos, porque para saber la población de un país simplemente escribimos ***pais["poblacion"]***, en vez de tener que acordarnos en qué posición está guardado ese dato.

## 1.3. Funciones

Una función es un bloque de código reutilizable que cumple una tarea específica. En vez de escribir el mismo código una y otra vez cada vez que lo necesitamos, lo definimos una sola vez dentro de una función, y después simplemente la "llamamos" cuando queremos usarla. Esto también ayuda a organizar el programa: cada función se encarga de una sola responsabilidad, lo que hace el código más ordenado y fácil de entender.

En nuestro proyecto dividimos el programa en varias funciones según su tarea: ***cargar\_paises*** se encarga de leer el archivo ***CSV***, ***agregar\_pais*** de dar de alta un nuevo país, ***actualizar\_paises*** de modificar los datos de un país existente, ***guardar\_paises*** de escribir los cambios en el archivo, y ***menu*** de mostrar las opciones y coordinar las llamadas a todas las demás funciones. De esta forma, si en algún momento necesitamos corregir o mejorar una parte del programa, sabemos exactamente en qué función buscar.

## 1.4. Estructuras condicionales

Las estructuras condicionales (***if***, ***elif***, ***else***) permiten que el programa tome decisiones según se cumpla o no una condición, ejecutando distintos bloques de código dependiendo del caso. En nuestro proyecto las usamos principalmente para validar datos ingresados por el usuario y manejar errores. Por ejemplo, en ***actualizar\_paises*** verificamos si el nombre ingresado está vacío:

```

if normalizar_nombre(nombre_buscado) == "":
    print("Error: el nombre no puede estar vacío.")
    return

```

Si la condición se cumple, el programa informa el error y termina la función con **return**, sin continuar con el resto del código. También usamos **if/elif/else** en el menú principal, para determinar qué opción eligió el usuario y ejecutar la función correspondiente, y dentro de los bucles **for** para comparar cada país con el nombre buscado.

## 1.5. Ordenamientos

Ordenar significa reorganizar los elementos de una lista según algún criterio, por ejemplo, de menor a mayor o al revés. En Python existe la función **sorted()**, que toma una lista y devuelve una nueva lista ordenada, sin modificar la original.

En nuestro proyecto, la función **ordenar\_paises** le pregunta al usuario por qué dato quiere ordenar (nombre, población o superficie) y si quiere que sea ascendente o descendente. Con esa información, se usa **sorted()** indicándole qué clave del diccionario tiene que mirar para comparar los países, y si el resultado tiene que ir al revés (**reverse=True**) o no. Así, sin tener que escribir nosotros mismos el algoritmo de ordenamiento, Python se encarga de reordenar toda la lista de países según el criterio elegido.

## 1.6. Estadísticas básicas

Son cálculos simples que se hacen sobre un conjunto de datos para obtener información resumida, como el promedio, el máximo o el mínimo. En nuestro proyecto, la función **mostrar\_estadisticas** recorre toda la lista de países con un **for** y va calculando varias cosas a la vez: busca cuál es el país con mayor y menor población (comparando uno por uno), suma todas las poblaciones y superficies para después calcular el promedio dividiendo por la cantidad total de países, y cuenta cuántos países hay por cada continente usando un diccionario donde la clave es el continente y el valor es la cantidad.

De esta forma, con un solo recorrido de la lista, se obtiene toda la información estadística pedida, sin tener que recorrer los datos varias veces para cada cálculo.

## 1.7. Archivos CSV

**CSV** significa “valores separados por comas”, y es un formato de archivo de texto muy simple que se usa para guardar datos en forma de tabla, donde cada línea representa una fila y los valores de cada columna van separados por comas. Es básicamente una versión muy básica de una planilla de cálculo.

En nuestro proyecto se utiliza el módulo **csv** de Python para trabajar con estos archivos. La función **cargar\_paises** usa **csv.DictReader**, que lee el archivo

***países.csv*** y convierte automáticamente cada fila en un diccionario, usando la primera línea (el encabezado) como nombres de las claves. Como todos los valores que vienen del CSV son ***strings***, se convierten población y superficie a números enteros con ***int()***. Por otro lado, la función ***guardar\_paises*** usa ***csv.DictWriter*** para hacer el proceso inverso: toma la lista de diccionarios y la escribe de nuevo en el archivo CSV, sobrescribiendo los datos anteriores con los actualizados.

## 2. Decisiones Técnicas y Arquitectura

### 2.1. Estructura de datos

Se decidió representar el conjunto de países como una lista de diccionarios. Cada diccionario representa un país con las claves nombre, poblacion, superficie y continente. Esta estructura fue elegida porque permite recorrer todos los países fácilmente con un bucle **for**, y acceder a cada dato de forma clara mediante el nombre de la clave, sin depender de posiciones fijas.

### 2.2. Modularización

El programa se dividió en funciones, cada una con una responsabilidad específica: carga y guardado de datos, alta y actualización de países, búsqueda, filtros, ordenamiento, estadísticas y el menú principal que coordina todo. Esta organización facilitó el trabajo en equipo, ya que cada integrante pudo desarrollar y probar sus funciones de forma independiente antes de integrarlas en un único programa.

## 2.3. Diagrama de flujo

A continuación, se presenta el diagrama de flujo de las operaciones principales del sistema:

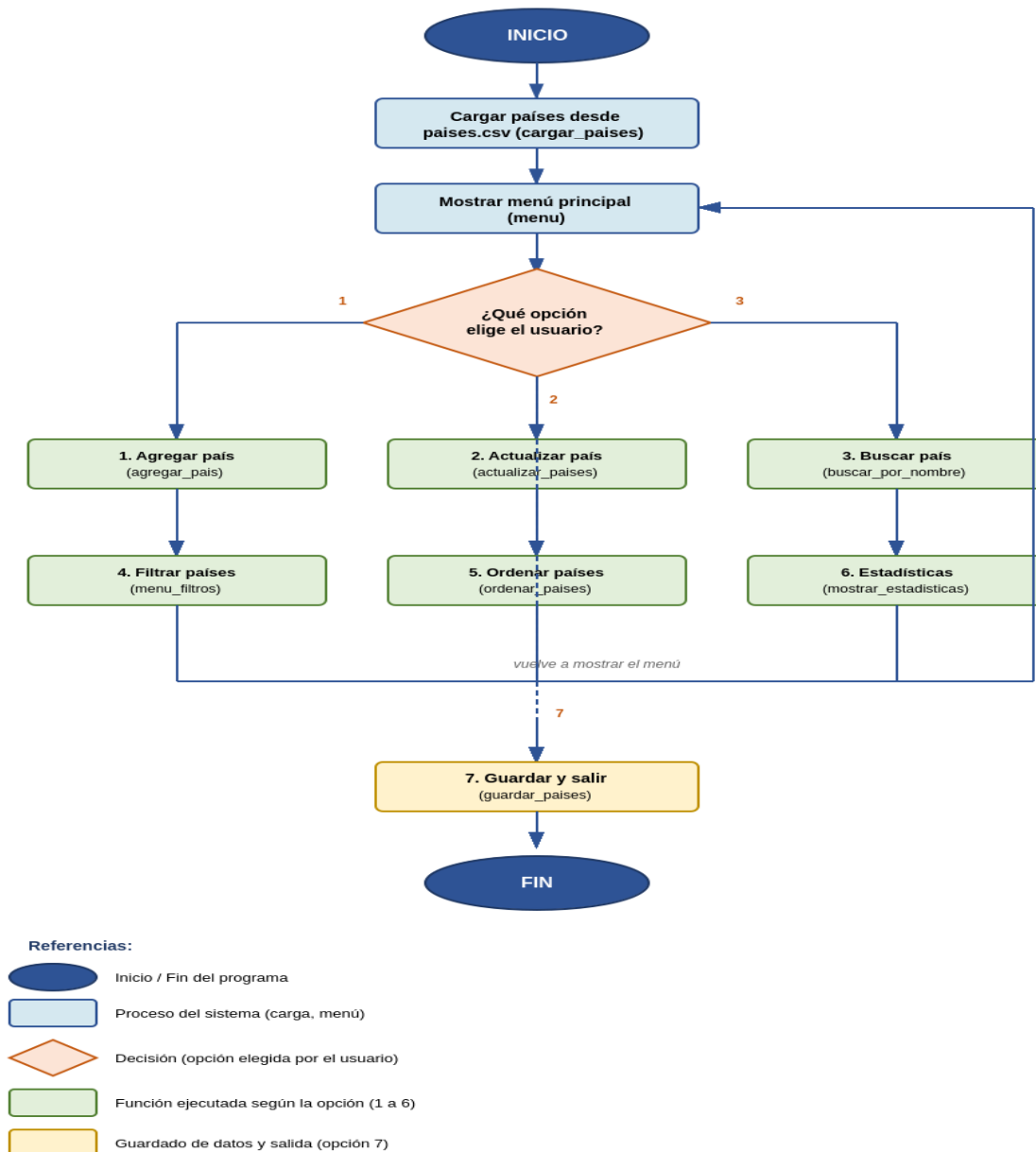


Imagen generada con asistencia de Claude, a partir de la información y estructura del programa desarrollado por el grupo.

## 2.4. Capturas de pantalla

A continuación, se muestran capturas de pantalla de la ejecución del sistema, incluyendo el menú principal y un ejemplo de cada una de las funcionalidades implementadas.

*Captura 1 - Menú principal*

```
--- GESTIÓN DE PAÍSES ---  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Estadísticas  
7. Guardar y salir  
Seleccione una opción: █
```

*Captura 2 - Opción 1 - Agregar país*

```
--- GESTIÓN DE PAÍSES ---  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Estadísticas  
7. Guardar y salir  
Seleccione una opción: 1  
Nombre del país: China  
Continente: Asia  
Cantidad de habitantes: 1  
Ingrese la superficie de China: 1  
'China' agregado correctamente.
```

*Captura 3 – Opción 2 – Actualizar país*

```
--- GESTIÓN DE PAÍSES ---  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Estadísticas  
7. Guardar y salir  
Seleccione una opción: 2  
Ingrese el nombre del país al que le quiere aplicar una actualización: China  
Nueva población de China: 1413022821  
Nueva superficie de China: 9596960  
'China' actualizado correctamente.
```



## Captura 4 – Opcion 3 – Buscar un país

```
--- GESTIÓN DE PAÍSES ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar y salir
Seleccione una opción: 3
Ingrese el nombre del país que desea buscar: argentina
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'America'}
```

## Captura 5 – Opcion 4 – Filtrar paises

```
--- GESTIÓN DE PAÍSES ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar y salir
Seleccione una opción: 4

--- FILTRAR POR ---
1. Continente
2. Rango de población
3. Rango de superficie
Opción: 1
Ingrese el nombre del continente que desea buscar: asia
{'nombre': 'Japon', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
{'nombre': 'China', 'poblacion': 1413022821, 'superficie': 9596960, 'continente': 'Asia'}
```

## Captura 6 – Opcion 5 – Ordenar paises

```
--- GESTIÓN DE PAÍSES ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar y salir
Seleccione una opción: 5

--- ORDENAR POR ---
1. Nombre
2. Población
3. Superficie
Criterio: 2
¿Orden ascendente? (S/N): s
{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'America'}
{'nombre': 'Francia', 'poblacion': 68042591, 'superficie': 551695, 'continente': 'Europa'}
{'nombre': 'Alemania', 'poblacion': 83149300, 'superficie': 357022, 'continente': 'Europa'}
{'nombre': 'Japon', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
{'nombre': 'Brasil', 'poblacion': 213993437, 'superficie': 8515767, 'continente': 'America'}
{'nombre': 'Nigeria', 'poblacion': 218541212, 'superficie': 923768, 'continente': 'Africa'}
{'nombre': 'China', 'poblacion': 1413022821, 'superficie': 9596960, 'continente': 'Asia'}
```

## Captura 7 – Opcion 6 – Estadísticas

```
--- GESTIÓN DE PAÍSES ---  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Estadísticas  
7. Guardar y salir  
Seleccione una opción: 6  
País con mayor población: China  
País con menor población: Argentina  
Promedio de población: 309703732  
Promedio de superficie: 3300512 km²  
Cantidad de países por continente:  
  America: 2  
  Asia: 2  
  Europa: 2  
  Africa: 1
```

## Captura 8 – Opcion 7 – Guardar y salir

```
--- GESTIÓN DE PAÍSES ---  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar países  
5. Ordenar países  
6. Estadísticas  
7. Guardar y salir  
Seleccione una opción: 7  
Datos guardados correctamente.
```

## 3. Dificultades y Conclusiones

### 3.1. Dificultades encontradas

El mayor problema apareció cuando juntamos nuestras dos partes en un solo archivo. Por un lado, en el menú quedaron líneas repetidas y mal indentadas, lo que hacía que el programa directamente no corriera. Por otro lado, las funciones de búsqueda, filtros, ordenamiento y estadísticas habían quedado escritas después de la función **menu()** y de la llamada que la ejecuta, entonces Python tiraba error porque todavía no las conocía (**NameError**). Lo solucionamos ordenando el archivo: arriba van todas las funciones, y al final, la carga del CSV y el llamado al menú. Esto nos sirvió para entender que Python lee el archivo de arriba hacia abajo, y que las funciones tienen que estar definidas antes de ser usadas.

También, nos costó un poco coordinarnos para trabajar los dos sobre el mismo repositorio. Al principio, uno de los integrantes terminó haciendo un **fork** (una copia separada del repositorio) en lugar de clonarlo, porque la invitación como colaborador todavía no había sido aceptada. Esto provocó que esos cambios no se vieran reflejados en el repositorio principal. Una vez aceptada la invitación, se pudo clonar el repositorio correcto y subir los cambios sin problema. Esta situación nos ayudó a entender mejor la diferencia entre tener permisos de colaborador en un repositorio y trabajar sobre un **fork** propio.

### 3.2. Conclusiones

Este trabajo nos resultó muy completo, ya que logró integrar prácticamente todos los conceptos que fuimos aprendiendo a lo largo de la materia: listas, diccionarios, funciones, condicionales, ordenamientos, estadísticas y manejo de archivos, todo aplicado a un mismo proyecto con sentido práctico.

Dividir el trabajo por funciones resultó una buena estrategia, ya que nos permitió avanzar en paralelo sin pisarnos demasiado, y entender con claridad qué hacía cada parte del programa al momento de integrarlo. El mayor desafío no estuvo tanto en lo técnico, sino en la coordinación: organizarse en equipo, repartir el trabajo de forma equitativa y mantener una comunicación clara durante todo el proceso. Sin embargo, creemos que logramos manejarlo bien, llegando a un resultado final en el que ambos entendemos y podemos explicar la totalidad del programa, no solo la parte que cada uno desarrolló.

En general, este TPI nos deja un aprendizaje que va más allá de lo técnico: además de afianzar los conceptos de Programación 1, nos ayudó a practicar el trabajo colaborativo con herramientas como GitHub, algo que seguramente vamos a seguir usando en el resto de la carrera.

## 4. Bibliografía / Webgrafía

1. UTN - Tecnicatura Universitaria en Programación (Modalidad a Distancia). Programación 1 - Unidad 7: Arreglos estáticos. Material de cátedra.
2. UTN - Tecnicatura Universitaria en Programación (Modalidad a Distancia). Programación 1 - Unidad 5: Funciones. Material de cátedra.
3. UTN - Tecnicatura Universitaria en Programación (Modalidad a Distancia). Programación 1 - Condicionales. Material de cátedra.
4. UTN - Tecnicatura Universitaria en Programación (Modalidad a Distancia). Programación 1 - Listas, Actividad 4. Material de cátedra.
5. UTN - Tecnicatura Universitaria en Programación (Modalidad a Distancia). Programación 1 - Apunte de Archivos. Material de cátedra.
6. Python Software Foundation. The Python Tutorial - 5. Data Structures. Documentación oficial de Python. Disponible en: <https://docs.python.org/3/tutorial/datastructures.html>
7. Python Software Foundation. csv - Lectura y escritura de archivos CSV. Documentación oficial de Python. Disponible en: <https://docs.python.org/es/3.8/library/csv.html>

## 5. Enlaces

**Repositorio GitHub:** <https://github.com/lucianoacq/tpi-paises-programacion1>

**Video demostrativo:** <https://www.youtube.com/watch?v=k1suDdM4U8I>